Technical University of Crete
School of Electrical and Computer Engineering
Course: Reinforcement Learning and Dynamic Optimization
Project: Optimal User Scheduling Using Deep RL Methods

Group 7 | Areti Xynompiti | Pavlos Sitoregko

1 Deep RL Method and Baselines

1.1 DQN Formulation

The goal of our agent is to learn which scheduling action is best in each state. For this reason, it approximates the function $Q(s, a)$, which measures the long-term effect of scheduling each user.

During training, actions are selected using an ϵ-greedy strategy for exploration. The value of ϵ is gradually decreased multiplicatively after each training episode according to: $\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon \cdot \epsilon_{\text{decay}})$, where ϵ_{decay} is chosen so that ϵ reaches its minimum value after 75% of the training episodes.

After each time step, the transition (s_t, a_t, r_t, s_{t+1}) is stored in a replay buffer. The agent samples mini-batches from this buffer, which improves training stability by reducing the correlation between consecutive samples. For each sampled transition, the target value is computed using the target network as:

$$y_t = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a')$$

The online network is trained by minimizing the mean squared error between the predicted value $Q_{\text{online}}(s_t, a_t)$ and the target value y_t. The update is performed using the Adam optimizer with learning rate $\alpha = 0.001$.

The target network weights are copied from the online network every 100 training steps, and since each training episode has a horizon of 200 slots, the target network is updated approximately twice per episode.

1.2 Baseline Scheduling Policies

Longest Queue. Schedule the user with the largest queue length:

$$a_t \in \arg\max_i q_i(t)$$

Best Non-Empty Channel. Schedule the non-empty user with the largest current service rate:

$$a_t \in \arg\max_{i:q_i(t)>0} r_i(t)$$

Greedy Throughput. Schedule the user with the largest immediate possible transmission:

$$a_t \in \arg\max_i \min(q_i(t), r_i(t))$$

Max-Weight. Schedule the non-empty user with the largest queue-service-rate product:

$$a_t \in \arg\max_i q_i(t) r_i(t)$$

2 Neural Network and Training Setup

2.1 State Representation

2.1.1 First naive try

The first input representation we considered was based on the queue length and the current service rate of each user, since these two quantities define the main state information available to the scheduler. In the smaller discrete scenario, one-hot encoding of the channel state was was efficient , because the number of channel states was limited. However, this representation is not as suitable in the larger setting, since it does not directly preserve the quantitative meaning of the values. In our problem, the magnitude matters: the neural network must understand not only that two rates are different, but also that one rate is higher or lower than the other. The same applies to the queues, since an empty queue and a nearly full queue represent very different scheduling situations.

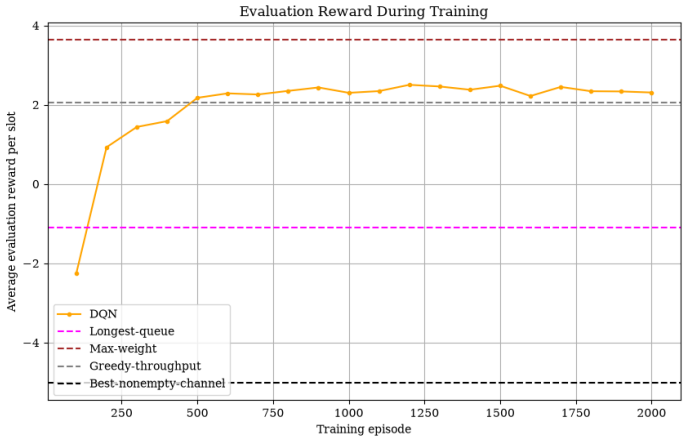
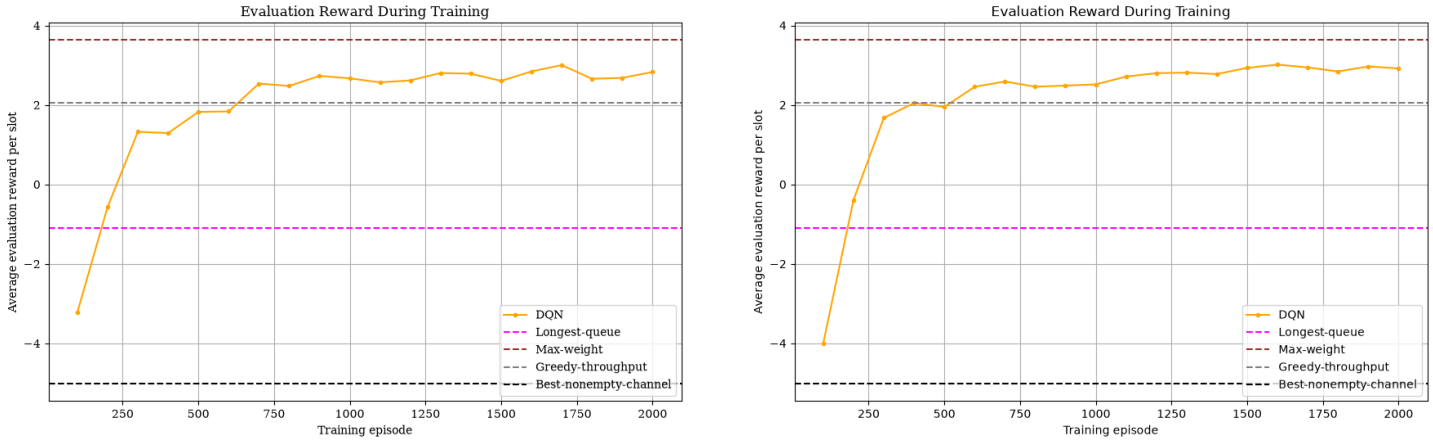


Figure 1: Rate hot-encoded and Raw Queue Values

2.1.2 Raw values as input vs normalized

Using the raw queue lengths and service rates as inputs was tested first, but the learning behavior was slower and a bit less stable. After normalizing, the DQN showed slightly more stable learning and reached good evaluation rewards earlier. However, the learned policy behaved somewhere between a channel-greedy policy and the Max-weight policy.



(a) Raw queue and rate input

(b) Normalized queue and rate input

Figure 2: Raw vs Normalized

2.1.3 Extra feature

This motivated us to examine what information the neural network might not easily infer from current inputs. For example, if a user has a very high service rate but only one packet in its queue, scheduling this user cannot use the full available rate. On the other hand, if another user has a large queue but a very low service rate, scheduling it may also give low immediate throughput. Therefore, neither the queue length nor the service rate alone is enough to describe how useful it is to schedule a user in the current slot.

For this reason, we added the possible transmission feature, defined as: $\min(q_i, r_i)$, which is the number of packets that can actually be transmitted if user i is selected. After adding this feature, the DQN achieved rewards identical to the Max-weight baseline.

2.1.4 Input features

The state is converted into a fixed-size input vector by concatenating three features for each user.

For user i, the input features are:

$\rightarrow$ the normalized queue length: $\dfrac{q_i(t)}{C}$

$\rightarrow$ the normalized service rate: $\dfrac{r_i(t)}{r_{\max}}$

$\rightarrow$ the normalized immediate possible transmission: $\dfrac{\min(q_i(t), r_i(t))}{r_{\max}}$

Therefore, the full state vector is: $s_t = \left[\dfrac{q_1(t)}{C}, \dfrac{r_1(t)}{r_{\max}}, \dfrac{\min(q_1(t), r_1(t))}{r_{\max}}, \ldots, \dfrac{q_N(t)}{C}, \dfrac{r_N(t)}{r_{\max}}, \dfrac{\min(q_N(t), r_N(t))}{r_{\max}}\right]$

2.2 Network Architecture

For the Q-network, we used a fully connected neural network with one hidden layer of 32 neurons and ReLU activation. This architecture was chosen experimentally. A smaller network with 16 neurons showed less stable learning, while a wider network with 64 neurons increased the runtime and gave slightly worse evaluation rewards. Since the 32-neuron network provided stable performance, we kept this simpler architecture and did not further increase the depth to two hidden layers.

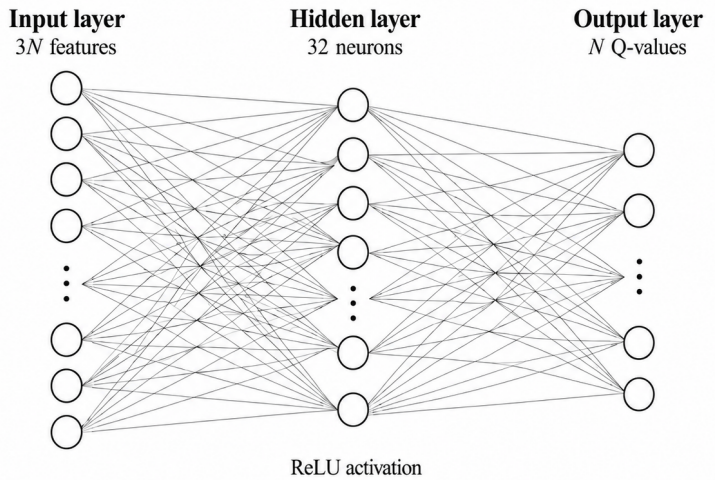


Figure 3: DQN neural network architecture

3 Results

3.1 Training Phase

To observe when DQN eventually learns, we evaluate the agent every 100 training episodes. Therefore, with a total of 1000 training episodes, the training phase produces 10 evaluation points.

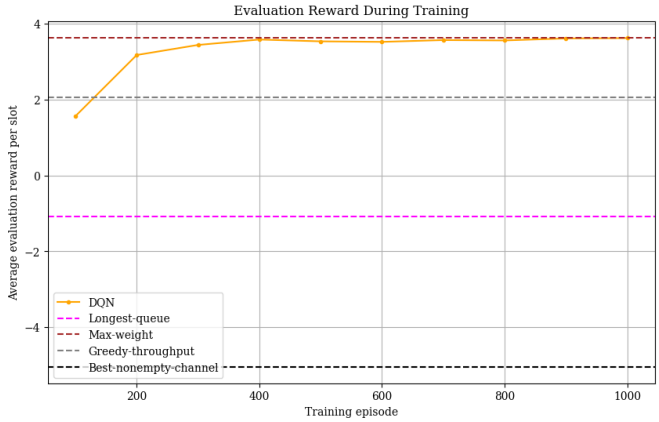


Figure 4: Evaluation reward of the DQN agent during training.

3.2 Evaluation Phase

Based on the training-phase results, the DQN learns after approximately 400–500 training episodes. Therefore, for the final evaluation, we train the agent continuously for 500 episodes and then perform a single evaluation phase. The final policy is evaluated over 1000 evaluation episodes, where each episode has a horizon of 500 time slots.

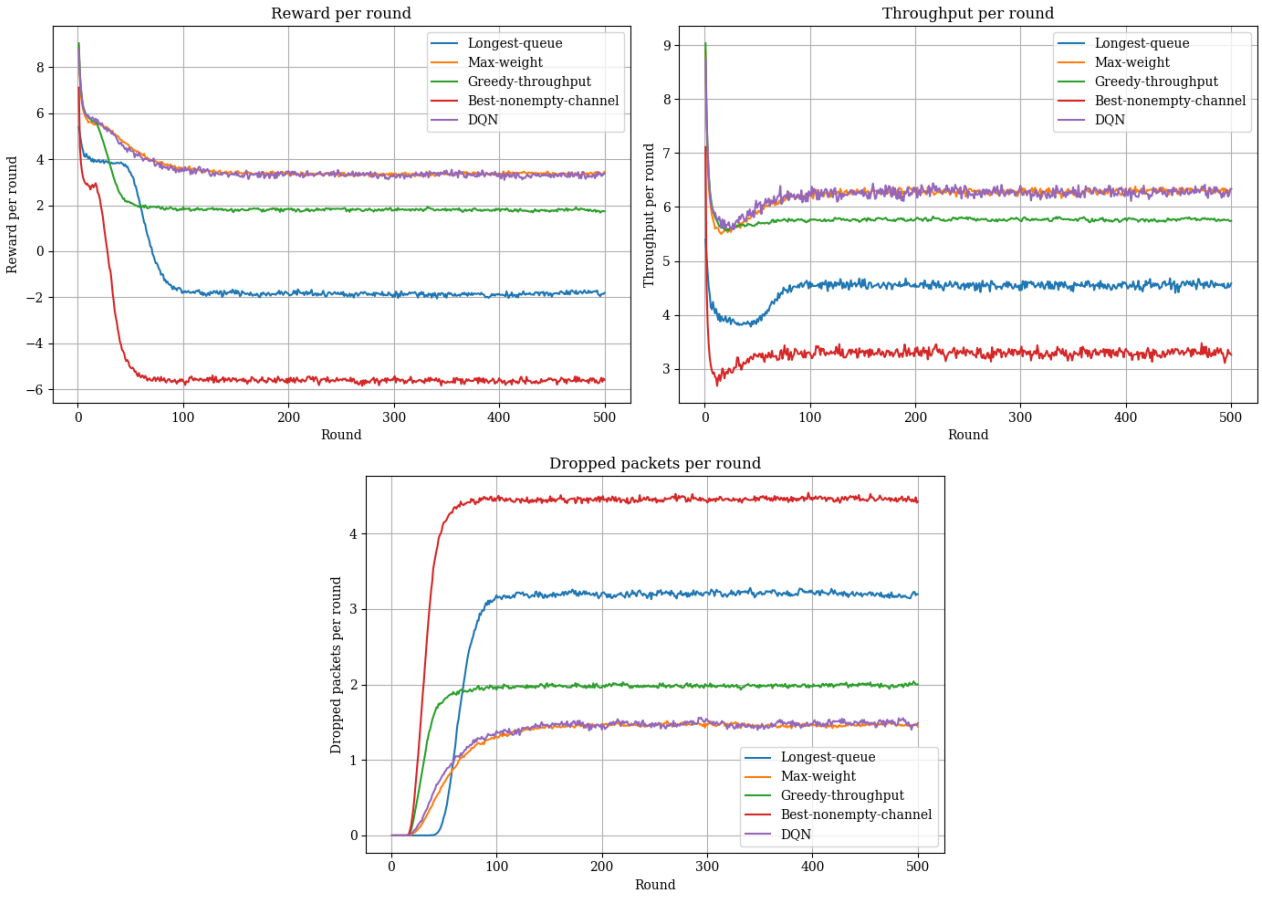


Figure 5: Evaluation performance of the DQN agent compared with the baseline scheduling policies.

The following results compare the final evaluation performance of the DQN policy and the Max-Weight baseline in terms of reward, throughput, and packet drops.

Metric	DQN	Max-Weight	Difference
Average reward per slot	3.594924	3.641485	-1.28%
Average throughput per slot	6.228396	6.222341	+0.10%
Average drops per slot	1.316736	1.290428	+2.04%

Table 1: DQN and Max-Weight evaluation results.

The bar plot that follows reports the mean queue occupancy for each user and the overall mean across all users during the evaluation phase.

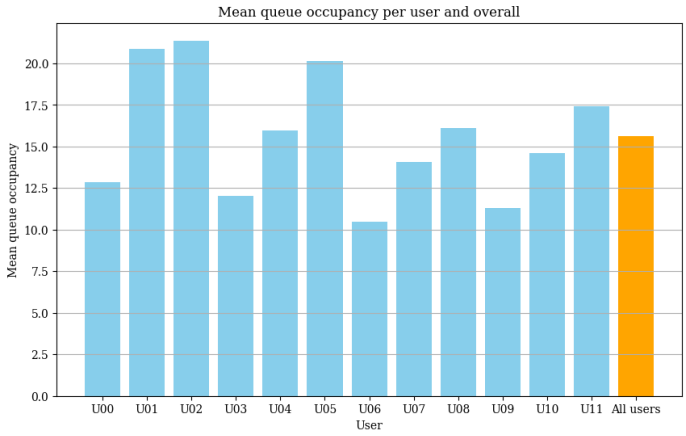


Figure 6: Mean queue occupancy per user and overall during evaluation.

4 Discussion: Max-Weight as a Near-Optimal Proxy

4.1 Proposed Benchmark: Look-ahead Beam-Search Heuristic

To obtain a stronger reference policy, we implement a look-ahead beam-search heuristic. The goal of this benchmark is to estimate how much performance can be gained when the scheduler is allowed to peak into the future, in contrast to the already implemented myopic baseline policies.

At each decision slot, the algorithm starts from the current queue state and expands possible future actions for a fixed number of look-ahead steps. Each candidate corresponds to an action sequence, the simulated queue state resulting from that sequence, and its cumulative predicted reward. The future service rates are taken directly from the trace, while arrivals inside the look-ahead simulation are replaced by their expected values. Therefore, the benchmark is deterministic during the look-ahead procedure and computationally cheaper than a Monte Carlo approach.

Since the full search tree grows exponentially with the look-ahead horizon, beam search is used to keep the computation manageable. After expanding all possible actions at each depth, the candidates are sorted according to their cumulative predicted reward, and only the best candidates are retained. In our implementation, the look-ahead horizon is set to $L = 3$, with beam widths $B_0 = 12$, $B_1 = 36$, and $B_2 = 108$ at the three depths. After the look-ahead procedure is completed, only the first action of the best-scoring sequence is executed in the real environment.

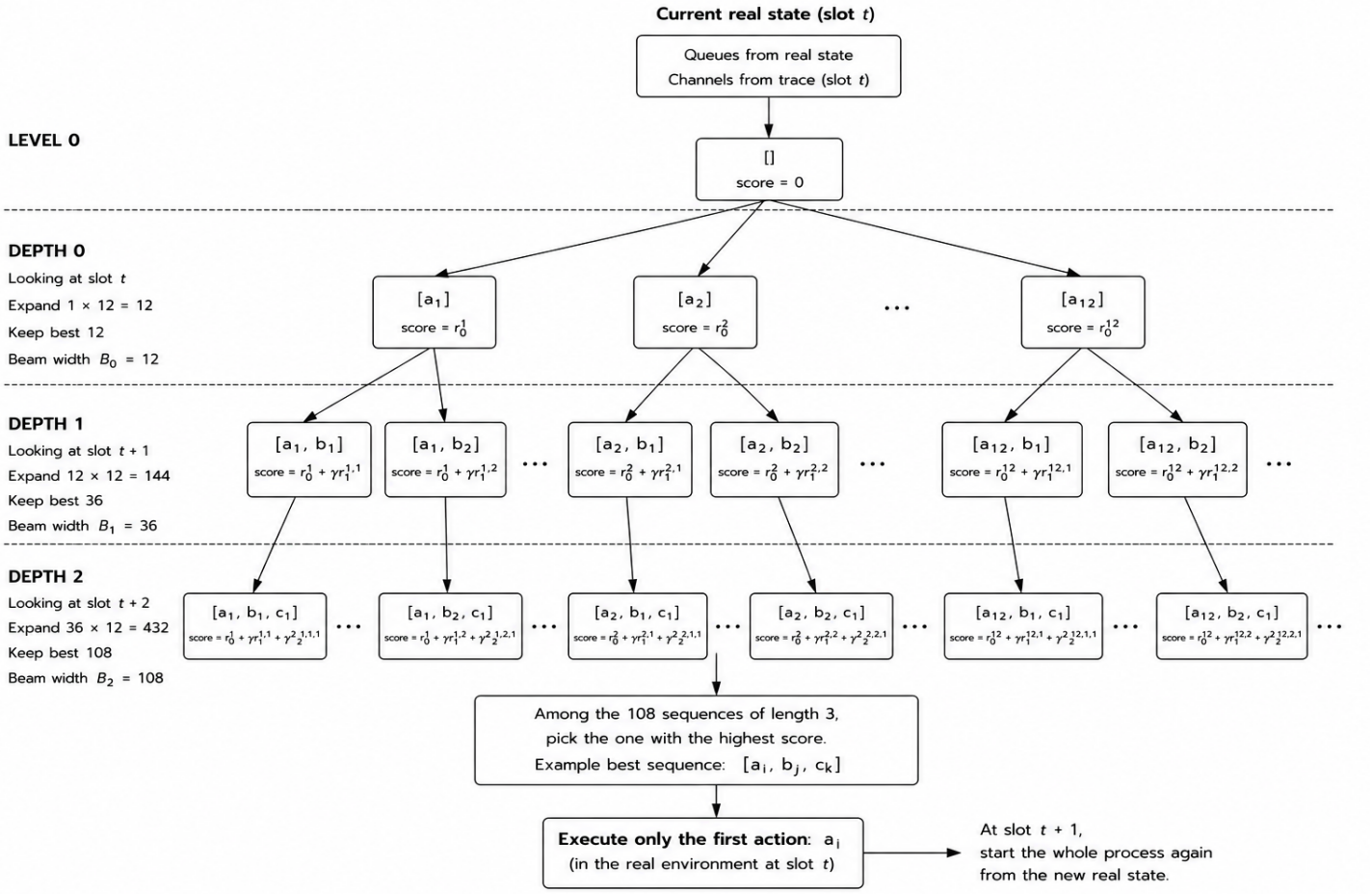


Figure 7: Beam-search look-ahead procedure for $L = 3$ with beam widths $[12, 36, 108]$

4.2 Results

In our results, the benchmark performs very close to Max-weight and DQN, without giving a clear additional gain. This suggests that, for this scenario, Max-weight policy already captures most of the useful scheduling structure. Thus, although DQN does not clearly outperform Max-weight, it successfully learns a competitive policy with similar reward, throughput, and drop behavior to the strongest benchmarks.

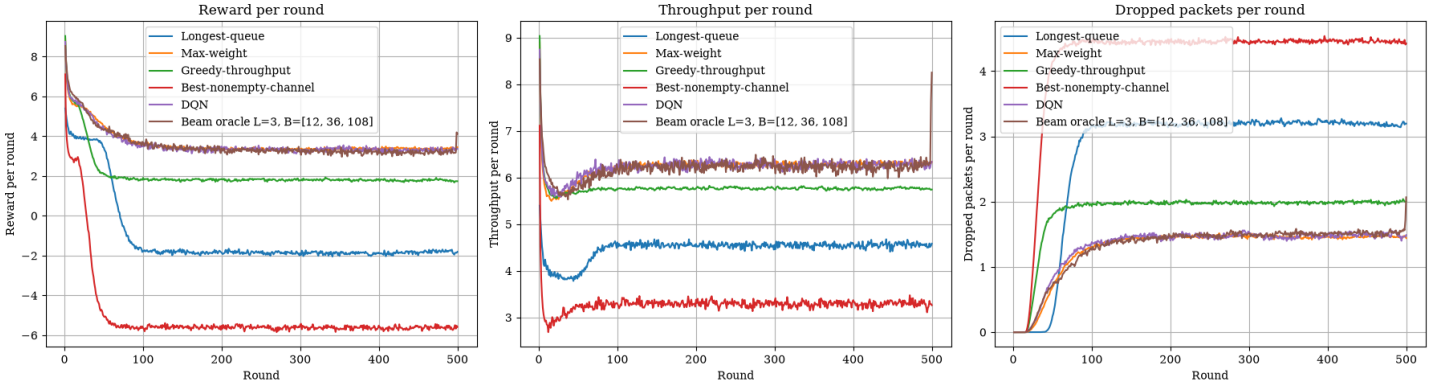


Figure 8: Baselines vs DQN vs Benchmark

Metric	DQN	Max-Weight	Look-ahead Beam-Search
Average reward per slot	3.594924	3.641485	3.574972
Average throughput per slot	6.228396	6.222341	6.193968
Average drops per slot	1.316736	1.290428	1.309498

Table 2: DQN, Max-Weight and Look-ahead Beam-Search evaluation results.

5 Computational Cost and Sample Efficiency

To quantify the sample efficiency of the DQN agent, we report the number of environment interactions used during training. The agent was trained for 500 episodes, with each training episode having a horizon of 200 time slots. Therefore, the total number of environment steps used for training was $500 \times 200 = 100000$ environment steps.

Training was performed locally on a machine equipped with an AMD Ryzen 5 7600 CPU. No GPU acceleration was used. No parallel environment simulation was used; episodes were generated sequentially.

The final policy was then evaluated over 1000 evaluation episodes, each with a horizon of 500 time slots. Therefore, the evaluation phase used $1000 \times 500 = 500000$ environment steps.

The wall-clock times for the DQN training and evaluation phases are shown below.

Phase	Wall-clock time (seconds)
Training	45.45
Evaluation	21.15

Table 3: DQN wall-clock times for training and evaluation.